



Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

**FACULTY OF INFORMATION AND COMMUNICATION
TECHNOLOGY**

**BACHELOR OF INFORMATION SYSTEMS (HONOURS)
INFORMATION SYSTEMS ENGINEERING**

UCCD3084 Graphics Programming for Extended Reality

June 2026 Trimester

Assignment Cover Sheet

Group 9 Pikachu Not Cute

No.	Name	ID	Course	Practical Group	Marks Awarded
1	Choong Khar Men	2305297	IA	P2	
2	Leong Chun Hui	2204619	IA	P1	
3	Lim Joo Phern	2206401	IA	P2	
4	Ng Choon Swee	2205754	IA	P2	

Table of Contents

CHAPTER 1: PROJECT SCOPE.....	1
1.1 Part A: Repair, Lightning and Optimization.....	1
1.2 Part B: Progress Tracking Display.....	3
CHAPTER 2: REPAIR & FEATURE DOCUMENTATION.....	4
2.1 Part A: Repair, Lighting & Optimization	4
2.1.1 Issue 1: UI — Onboarding Fade Screen	4
2.1.2 Issue 2: Toggle Logic — Console Labels.....	5
2.1.3 Issue 3: Lighting — Factory Floor.....	6
2.1.4 Issue 4: Lighting — Box Piles.....	7
2.1.5 Issue 5: Lighting — Console	7
2.1.6 Issue 6: Visual Feedback – Magnet State	8
2.1.7 Issue 7: Interaction – Instruction Lock	9
2.2 Part B: Additional Interactive Feature (Progress Tracking Display).....	10
2.2.1 Purpose of Progress Tracking Display.....	10
2.2.2 User Interface Elements of Progress Tracking Display.....	10
2.2.3 Behavior and Implementation.....	11
CHAPTER 3: SYSTEM FLOW DIAGRAM.....	12
CHAPTER 4: TEAM CONTRIBUTION	14
CHAPTER 5: TESTING & RESULTS	15
CHAPTER 6: SCRIPTS USED FOR FACTORY VR TRAINING SIMULATION.....	17

CHAPTER 1: PROJECT SCOPE

The goal of this project is to combine Unity and the XR Interaction Toolkit to create a useful and engaging virtual reality training simulation situated in a factory. The project starts with a purposefully damaged factory prototype that has problems with interaction control, lighting, visual feedback, and user interface. The scope is split into two primary sections: Part A, which concentrates on fixing and improving the current manufacturing environment, and Part B, which adds an extra interactive element known as the Progress Tracking Display.

1.1 Part A: Repair, Lightning and Optimization

Part A covers the diagnosis and repair of the major issues found in the original factory prototype. While maintaining the original factory layout and XR rig, these modifications aim to enhance the VR experience's usability, visual quality, performance, and realism.

The first repair addresses the onboarding fade screen. Instead of obscuring the user's view, the original fade-in canvas was positioned wrongly within the factory environment. When the application launches, the canvas and camera settings are set so that the user's entire field of view is covered by the black fading screen.

The second repair improves the Console Labels toggle. Previously, activating the toggle caused the entire console to disappear. The main console and its controls are still visible and functional, but the toggle logic has been adjusted so that only the instructional labels are hidden or displayed.

The third repair focuses on the factory floor lighting. At first, the floor seemed less detailed and flatter than the surrounding surfaces. The lightmap parameters are changed to enhance the floor's visual quality and offer more consistent baked lightning detail.

The fourth repair enhanced the lightning and shadows of the box piles. The box piles are configured correctly for lightmap baking. As a result, all of the factory's box piles have more realistic and uniform shadows.

The fifth repair addresses the lightning of the control console. Since the overhead lightning remains baked for optimization, light probe settings are used to ensure the console receives appropriate indirect lightning and no longer appears excessively dark.

The sixth repair adds visual feedback to the magnet mechanic. The magnet uses different visual states to indicate whether it is active or inactive. In order to give further feedback when the magnet is operated, a particle effect is added to the magnet's operation.

The last repair introduces an interaction lock during onboarding. The user cannot operate the console while the instructions are shown. In order to improve the clarity and management of the interaction flow, the user's ray and direct interaction indications are deactivated once the instructions have been finished.

In addition to these seven repairs, the overall VR experience includes teleportation movement, snap turning, a vignette comfort option, spatialized audio, and a reset mechanism. These features facilitate comfortable navigation and reset the scene to its initial state without reloading the scene.

1.2 Part B: Progress Tracking Display

The additional interactive feature in this project is the Progress Tracking Display. This feature demonstrates a visual summary of the trainee's performance during the factory training session. A progress bar, total target number of boxes, the amount of training time, and the number of boxes successfully lifted by the magnet are all displayed on the screen.

This feature is connected to the existing box-lifting mechanism. When a trainee successfully lifts a box, the system updates the progress count and visual progress bar. To avoid making duplicate data, each box is only counted once. When the training session begins, the timer starts, and it stops when the required number of boxes has been lifted. A reset button has been provided to allow the progress count and timer to reset to their initial values.

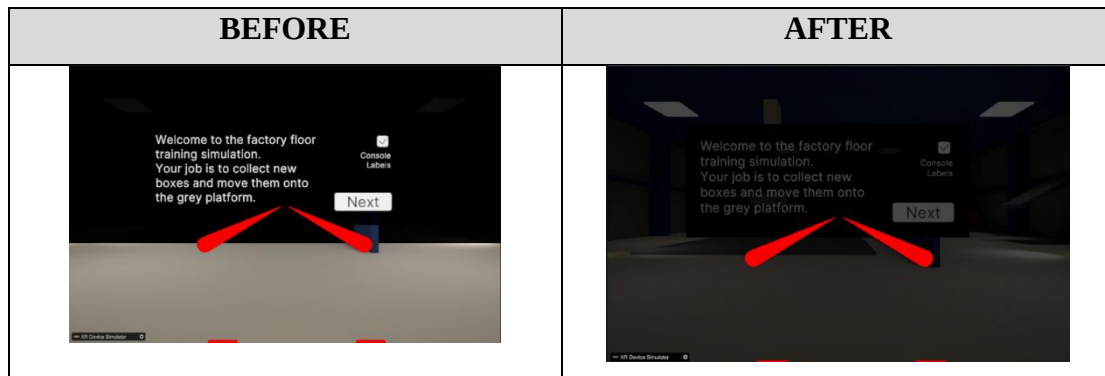
This feature was chosen since there was no clear way for trainees to track their progress or evaluate their performance. A progress display provides instant feedback during interaction, and helps trainee understand how many boxes are left. By displaying the completion status, it also increases the trainee's motivation.

Since it builds directly on the current magnet interaction and demonstrates the use of VR interaction events, contextual UI, feedback systems, and session management, this feature is appropriate for the factory training scenario. It enhances the training experience and guaranteeing that the interaction is comfortable and easy to understand.

CHAPTER 2: REPAIR & FEATURE DOCUMENTATION

2.1 Part A: Repair, Lighting & Optimization

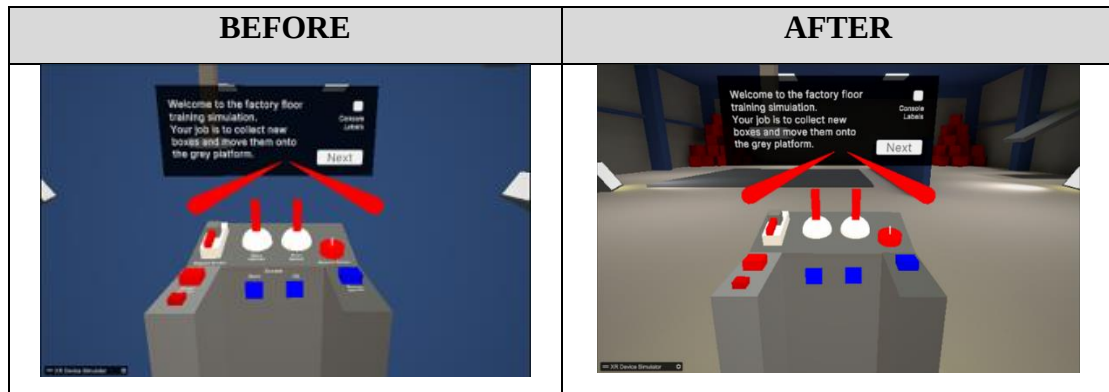
2.1.1 Issue 1: UI — Onboarding Fade Screen



Before the repair, the onboarding fade image did not cover the complete camera view. The lower part of the factory remains visible, which weakened the intended fade-in transition.

To solve this issue, the **Canva_Fade** render mode was changed to **Screen Space – Camera** and its Render Camera was assigned to the **Main Camera**. This ensures that the fade interface follows the player's camera instead of appearing as an object positioned inside the 3D environment. The Canvas Scaler was configured to **Scale With Screen Size** with a reference resolution of **1920 × 1080**, allowing the interface to resize correctly for different display resolutions. The child fade Image was then set to stretch across the entire canvas by using full-stretch anchors and setting all offsets—Left, Right, Top, and Bottom—to zero. Consequently, the black image covers the complete camera viewport during the fade transition, as shown in second image above, with no visible floor or uncovered edges.

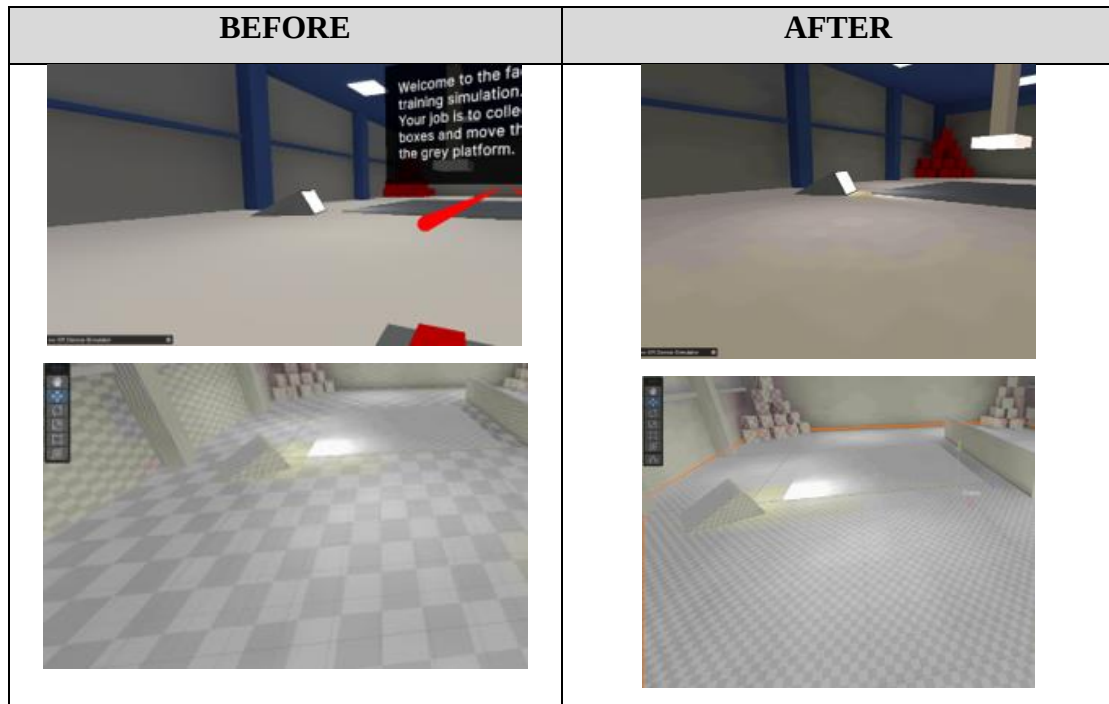
2.1.2 Issue 2: Toggle Logic — Console Labels



The original Console Labels toggle was connected to the complete console GameObject. Therefore, clearing the checkbox disabled the entire console, including its buttons, levers and other controls instead of hiding only the instructional labels.

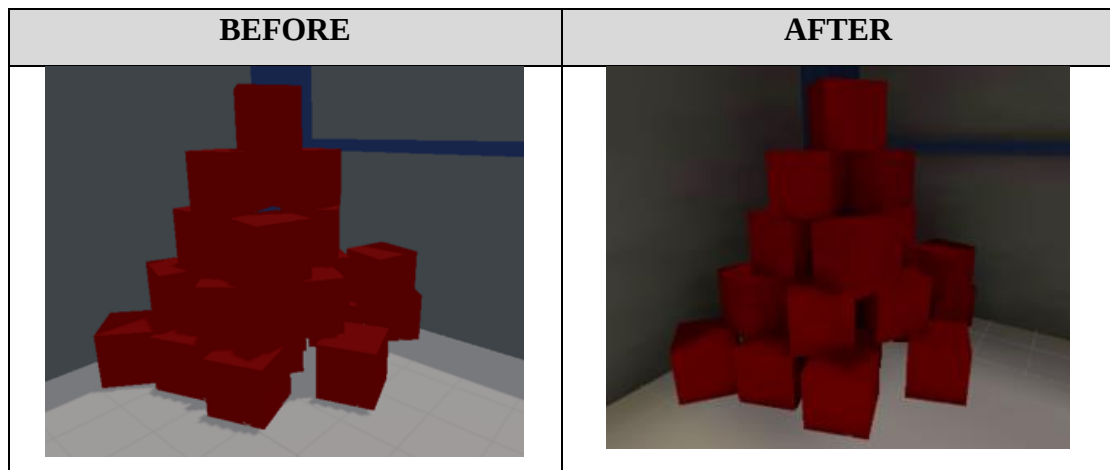
To fix the issue, the Toggle component's On Value Changed (Boolean) event was reassigned from the complete console to the Canvas_Labels GameObject. The event was configured to all the dynamic GameObject.SetActive (bool) function. This dynamic Boolean value follows the checkbox state which when the checkbox is selected, Canvas_Labels is activated and the instructional labels are displayed; when the checkbox is cleared, Canvas_Labels is deactivate and the labels are hidden. Because only the label canvas is controlled by the event, the physical console, buttons and levers remain active, visible and usable in both states.

2.1.3 Issue 3: Lighting — Factory Floor



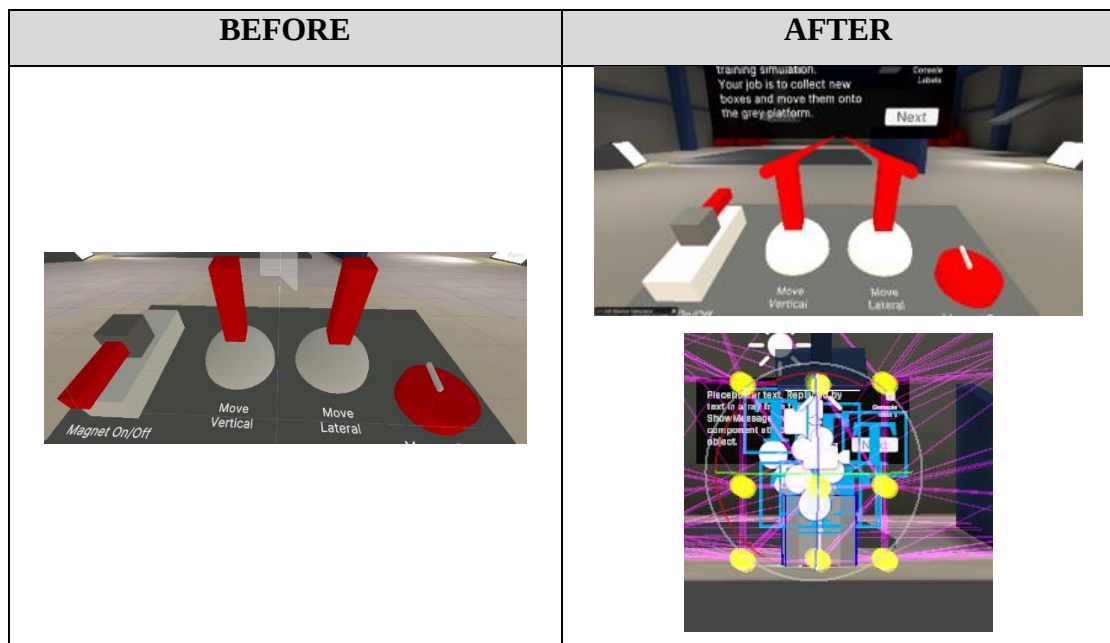
The scale of light map previously is 0.1, which made the lighting on floor less detailed. Within the floor object's Mesh Renderer settings, the **scale in lightmap** parameter was increased to **1.0**, increasing the relative texel density assigned to the surface. Global Illumination static lightmaps were then completely regenerated using the Unity Lighting Manager. This adjustment allowed the baking engine to calculate fine-grained lighting transitions and crisp contact shadows, resulting in significantly enhanced visual fidelity across the walkable area without unnecessarily bloating lightmap memory for smaller scene props.

2.1.4 Issue 4: Lighting — Box Piles



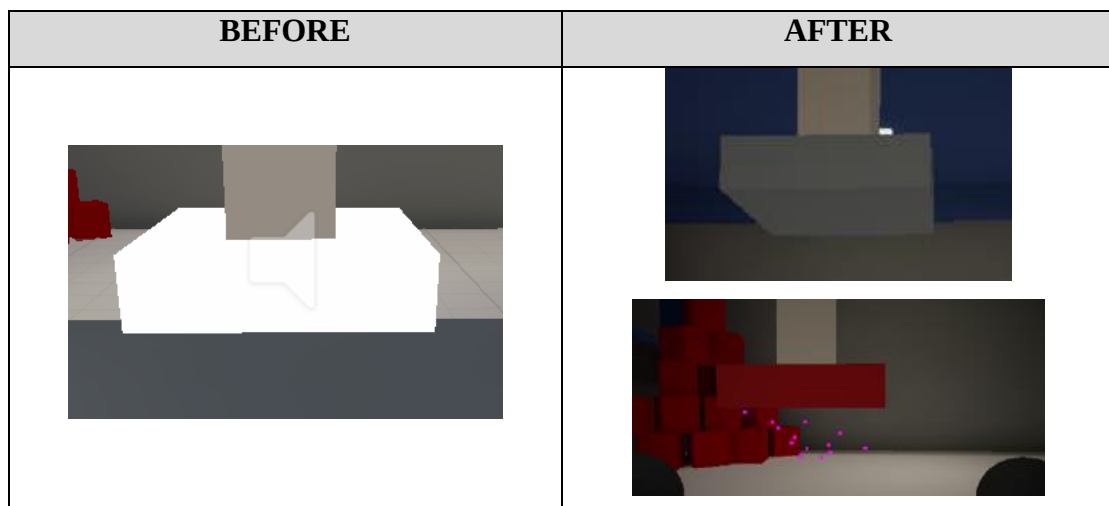
Since the pile of boxes in the back left corner was not designated as static, that is why the shadow and lighting look unrealistic. In Unity, static is required to check in order to baking the lighting in the game object. Marking the box group as Static enables the light mapper to calculate realistic soft shadows and environmental color bounce across the geometry. Re-baking the scene after enabling static flags anchors the props into the environment visually while preserving real-time rendering performance.

2.1.5 Issue 5: Lighting — Console



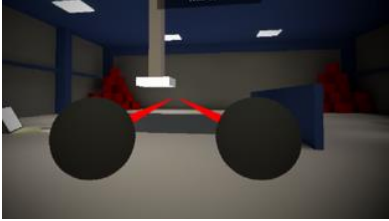
Since non-static or interactive Game Objects cannot receive 2D baked lightmaps, the console remains dark despite the static light positioned above it. Adding a **Light Probe Group** solves this optimization challenge by capturing and storing baked light intensity, direction, and color at discrete 3D points in space. By surrounding the console with probes and regenerating the lightmap, the console's **Mesh Renderer** automatically samples lighting data from the nearest probes using spherical harmonics. This allows the console to reflect the overhead baked light realistically without converting the light to expensive real-time rendering or requiring the interactive console to be marked static.

2.1.6 Issue 6: Visual Feedback – Magnet State



Before the repair, the magnet did not give a clear visual representation of its current condition. Trainees found it difficult to tell if the magnet was active or not. To fix the issue, two different materials: `InactiveMagnet` and `ActiveMagnet` material, were configured for the magnet. The magnet also enhanced its visual representation by using a particle system called `MagneticParticles`. To make the visual representation works, the magnet control is used to configure the activation and deactivation events. The system initiates the particle effect, and switches the magnet material to the active material when it is activated. When deactivated, the particle effect is stopped, the original material is restored.

2.1.7 Issue 7: Interaction – Instruction Lock

BEFORE	AFTER
	

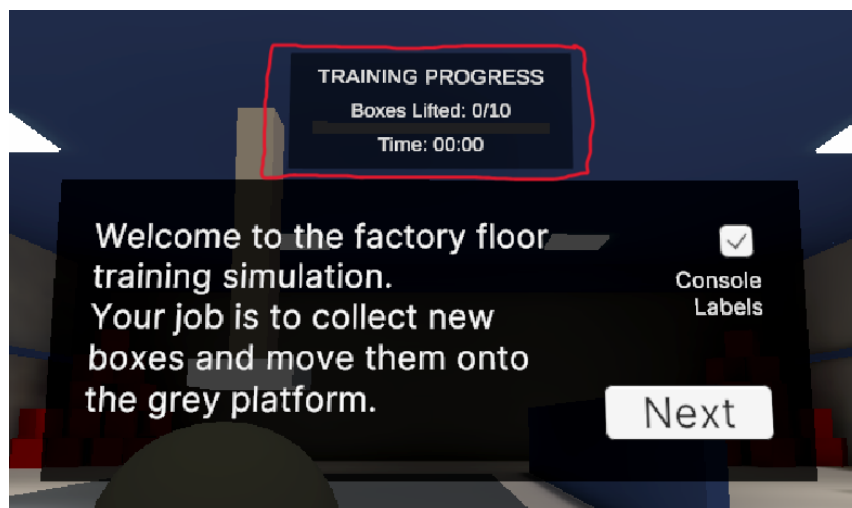
The original SceneLoad configuration only handled player teleportation and the fade-in effect. It did not include any functions to disable the left and right ray. As a result, user activity was not appropriately restricted by the onboarding screen. To fix this issue, the ToggleRay.DeactivateRay events were added for both LeftRay and RightRay. After the trainee has finished the onboarding instructions, these events control the ray status and return to the proper interaction mode.

2.2 Part B: Additional Interactive Feature (Progress Tracking Display)

2.2.1 Purpose of Progress Tracking Display

The Progress Tracking Display provides real-time visual and audio feedback during the box-delivery task. It displays the number of boxes successfully delivered, the elapsed training time, the overall completion status and a progress bar. The feature helps trainees understand their remaining workload and confirms whether each interaction has been accepted or rejected by the system.

2.2.2 User Interface Elements of Progress Tracking Display



Based on the image, the Progress Tracking Display includes **7 components**:

Component	Function
Canvas_Progress	Holds the world-space progress interface.
BoxesText	Shows the accepted box count out of 10.
TimerText	Shows elapsed time in MM:SS format.
TitleText	Fills according to count divided by 10.
ProgressBarFill	Shows normal, success, duplicate and invalid states.
ProgressFeedbackAudio	Plays correct, incorrect and completion sounds.
ProgressSuccessParticles	Produces successful-delivery and completion bursts.

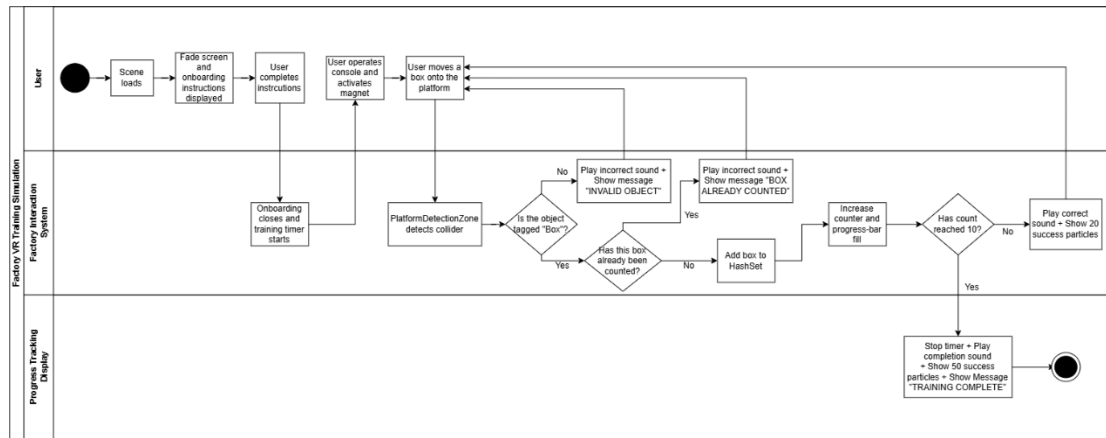
2.2.3 Behavior and Implementation



When the onboarding instructions are completed, `StartTimer()` begins the training timer. A box is counted when it enters the `PlatformDetectionZone`. The collider is passed to `RegisterPlatformBox()`, which first checks the `Box` tag and then uses a `HashSet` to prevent the same box from being counted twice. A first-time valid delivery increments the counter and updates the progress bar. It also produces a correct audio cue, a green temporary status message and a 20-particle burst. Invalid objects and repeated boxes are rejected with different status colours and an incorrect audio cue. When the tenth box is accepted, the timer stops, the completion audio plays, 50 particles are emitted and the display changes to “TRAINING COMPLETE!”

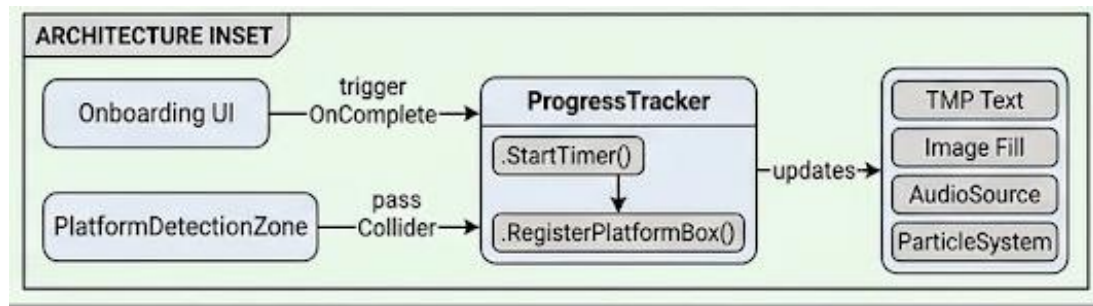
CHAPTER 3: SYSTEM FLOW DIAGRAM

Swimlane Flowchart for Factory VR Training Simulation:



The swimlane flowchart illustrates the complete interaction flow of the factory training simulation. After the onboarding instructions are completed, the timer starts and the trainee can operate the console and magnet. When an object enters the platform detection zone, the system validates whether it is a box and whether it has already been counted. Valid boxes update the counter and progress bar, while invalid or duplicate objects produce negative feedback. The process continues until ten boxes are delivered, after which the timer stops and the completion feedback is activated.

Architecture Inset for Factory VR Training Simulation:



The architecture inset shows how the main Unity components communicate. The onboarding interface calls `ProgressTracker.StartTimer()` when the instructions are completed. The `PlatformDetectionZone` passes the detected collider to `RegisterPlatformBox()`. The `ProgressTracker` then updates the TextMeshPro interface, progress-bar image, audio source and particle system according to the validation result.

CHAPTER 4: TEAM CONTRIBUTION

Group Member	Task Contribution
Choong Khar Men	- Part B: Progress Tracking Display - Combine the group report
Leong Chun Hui	- Part A: Visual Feedback – Magnet State - Part A: Interaction – Instruction Lock - Part A: Interaction – Instruction Lock - Merge & verify files in GitHub to avoid conflict
Lim Joo Phern	- Part A: Visual Feedback – Magnet State - Part A: Interaction – Instruction Lock - Configure initial group GitHub setup
Ng Choon Swee	- Part A: UI — Onboarding Fade Screen - Part A: Toggle Logic — Console Labels - Combine the demonstration video

CHAPTER 5: TESTING & RESULTS

Testing was conducted in Unity 2022.3.62f3 using the Training_Prototype_Broken scene, XR Interaction Toolkit 2.6.4, and Meta Quest 2 (test in VR headset) or XR Device Simulator (test in computer).

By using table:

Test Cases	Feature Tested	Test Procedure	Expected Result	Final Result
T01	Onboarding fade	Start the simulation	Black fade covers the complete view and clears smoothly	Pass
T02	Console labels	Toggle labels off and on	Labels change visibility while the console remains visible and usable	Pass
T03	Floor lighting	Inspect the floor under baked lighting	Improved detail with consistent lighting and contact shadows	Pass
T04	Box-pile lighting	Inspect all static box piles	Realistic, consistent baked shadows without floating appearance	Pass
T05	Console lighting	View and operate the console	Console receives appropriate indirect lighting from probes	Pass
T06	Magnet feedback	Activate and deactivate magnet	Material, particles and audio correctly represent its state	Pass
T07	Instruction lock	Try interacting during onboarding, then complete it	Controls are unavailable during onboarding and restored afterward	Pass

T08	Progress display	Deliver valid, duplicate and invalid objects; reach 10 boxes	Valid boxes count once; incorrect entries are rejected; timer and bar update; completion occurs at 10	Pass
T09	Locomotion	Test teleportation, snap turn and vignette	Each locomotion feature works without unwanted movement or visual obstruction	Pass
T10	Reset	Make progress, move boxes and activate magnet; trigger reset	Boxes/player/magnet reset; counter becomes 0/10; timer becomes 00:00; feedback clears	Pass

CHAPTER 6: SCRIPTS USED FOR FACTORY VR TRAINING SIMULATION

1. ToggleParticle.cs

ToggleParticle.cs manages the particle system attached to a Unity GameObject. Play() is used to start the particles, while Stop() is used to stop them. When the magnet becomes active, the script displays the magnetic particles, and when the magnet is inactive, it hides them. This feature is to enhance the visual representation on the mode the magnet.

2. ProgressTracker.cs

ProgressTracker.cs manages the Progress Tracking Display for the factory training simulation. When the trainee completes the onboarding instructions, StartTimer() begins recording the elapsed training time. When an object enters the platform detection zone, RegisterPlatformBox() checks whether the object has the Box tag and whether it has already been counted. A valid box increases the delivered-box counter and updates the progress bar, while a HashSet prevents the same box from being counted more than once. The script also controls the system's visual and audio feedback. A valid delivery produces a green status message, a correct audio cue and a particle burst. Invalid objects and duplicate boxes produce different warning messages and an incorrect audio cue. When all ten boxes have been delivered, CompleteTraining() stops the timer, displays "TRAINING COMPLETE!", plays the completion sound and emits a larger particle burst. The ResetProgress() function returns the counter, timer, status message, audio and particles to their initial states.